

Doc. no.	P0024R2
Date:	2016-03-04
Project:	Programming Language C++
Reply to:	Jared Hoberock < jhoberock@nvidia.com >

The Parallelism TS Should be Standardized

Abstract

We survey implementation experience with the recently published C++ Technical Specification for Extensions for Parallelism and conclude that ample experience with its functionality exists to justify standardization in C++17. This paper describes various existing and pre-existing implementations of the TS's content and describes the additions to be made to the current C++ working paper (N4527) to integrate execution policies and parallel algorithms into the C++ Standard Library.

Implementation Experience

Although the first version of the Parallelism TS has been published only recently (N4354), we believe practioners have suitable experience with both existing and pre-existing implementations of its functionality to allow prompt standardization. Both implementors and users are ready for parallel algorithms in C++.

Development History

The technical content of the Parallelism TS was developed by domain experts in parallelism over the course of a few years. In 2012, representatives from NVIDIA (N3408) as well as representatives from Microsoft and Intel (N3429) independently proposed library approaches to parallelism within the C++ Standard Library. At the suggestion of SG1 the authors of these proposals submitted a design in a joint proposal (N3554) to parallelize the existing standard algorithms library. This proposal was refined into the Parallelism TS over the course of two years. During that refinement process, the authors of the Parallelism TS incorporated feedback from experimental implementations into the final design which was published in 2015. In total, the C++ Standardization Committee has three years of experience with the TS's design.

Existing implementations

Several different implementations of the Parallelism TS emerged during its preparation. We are aware of the following publically documented implementations.

- Microsoft: <http://parallelstl.codeplex.com>
- HPX: <http://stellar-group.github.io/hpx/docs/html/hpx/manual/parallel.html>
- Codeplay: <http://github.com/KhronosGroup/SyclParallelSTL>
- HSA: <http://www.hsafoundation.com/hsa-for-math-science>
- Thibaut Lutz: <http://github.com/t-lutz/ParallelSTL>
- NVIDIA: <http://github.com/n3554/n3554>

These implementations implement the functionality of the Parallelism TS to varying degrees and in different ways. For example, Microsoft's implementation appears complete and is implemented via Windows-specific tasking facilities. Thibaut Lutz' version also appears complete and is implemented by manipulating `std:::thread` in a standard way. NVIDIA's implementation is partial and is implemented as a thin wrapper around Thrust, a pre-existing library similar in content to the Parallelism TS. This variety of implementation approaches exists by design: the abstractions of the Parallelism TS are intended to maximize flexibility of implementation.

Pre-existing implementations

The design of the Parallelism TS's functionality was inspired by several pre-existing libraries. Each of the following parallel algorithms libraries expose an iterator-based algorithm interface based on the conventions of the original Standard Template Library. We believe these libraries are a reasonable proxy for the content of the Parallelism TS.

- Thrust: <http://thrust.github.io>
- Boost.Compute: <http://github.com/boostorg/compute>
- Bolt: <http://github.com/HSA-Libraries/Bolt>
- libstdc++ Parallel Mode: http://gcc.gnu.org/onlinedocs/libstdc++/manual/parallel_mode.html
- AMP Algorithms Library: <http://ampalgorithms.codeplex.com>

These libraries have existed for several years, and some are widely deployed in production. Accordingly, we believe the features of the Parallelism TS are proven abstractions that represent standard practice and solve real challenges faced by real C++ programmers. These challenges exist because parallel architectures are so pervasive, and programming them correctly with existing low-level standard components is difficult. As a remedy, we believe that the high-level abstractions of the Parallelism TS must be standardized as soon as possible. C++ programmers should not have to wait beyond 2017 for standard parallel algorithms.

Future Support

The parallel algorithms and execution policies of the Parallelism TS are only a starting point. Already we anticipate opportunities for extending the Parallelism TS's functionality to increase programmer flexibility and expressivity. A fully-realized executors feature (N4414, N4406) will yield new, flexible ways of creating execution, including the execution of parallel algorithms. For example, executors will provide a programmatic means of specifying *where* execution is allowed to occur during parallel algorithm execution and will open the door for user-defined execution policies in addition to the Parallelism TS's closed set of standard policies. If the first version of the Parallelism TS is standardized in 2017, such additional features for parallelism will be well-positioned for 2020.

Summary of proposed changes

We propose to standardize the functionality of the Parallelism TS as specified. In summary:

- Add `exception_list` as a new subclause to Clause 19.
- Add execution policies as a new subclause to Clause 20.
- Introduce general wording for parallel algorithms as a new subclause to Clause 25.
- Add the new algorithms introduced by the Parallelism TS to Clauses 20, 25, and 26.

- Add header <execution_policy> to Table 14.

Acknowledgements

Thanks to Jens Maurer for wording suggestions.

References

1. N4308 - [Parallelizing the Standard Algorithms Library](#), J. Hoberock, M. Garland, O. Giroux, V. Grover, U. Kapasi, and J. Marathe. 2012.
2. N4329 - [A Library Solution to Parallelism](#), A. Laksberg, H. Sutter, A. Robison, and S. Mithani. 2012.
3. N3554 - [A Parallel Algorithms Library](#), J. Hoberock, J. Marathe, M. Garland, O. Giroux, V. Grover, A. Laksberg, H. Sutter, and A. Robison. 2013.
4. N4354 - [Programming Languages - Technical Specification for C++ Extensions for Parallelism](#), International Standards Organization. 2015.
5. N4414 - [Executors and schedulers, revision 5](#), C. Mysen. 2015.
6. N4406 - [Parallel Algorithms Need Executors](#), J. Hoberock et al. 2015.

Changelog

- P0024R0
 - Initial document.
- P0024R1
 - Eliminate greenlining.
 - Eliminate anachronistic use of TS-specific namespaces.
 - Eliminate NaN-numbered sections and number sequentially.
 - Introduce Acknowledgments section.
 - Eliminate unwarranted explanations in for_each's notes.
 - Use the term "unspecified" in reduce's note.
 - Replace use of "sequential order" with "indeterminately sequenced" in S 25.4p2.
 - Eliminate the "in an unordered fashion" phrase (which was redundant and undefined) from S 25.4p3.
 - Add a cross-reference to (1.9) in S 25.4p3.
- P0024R2
 - Eliminate greenlining.
 - Eliminate dynamic execution_policy class.
 - Rename seq to sequential.
 - Incorporate LWG's guidance on wording.

Contents

20	General utilities library
20.15	Execution policies
20.15.1	In general
20.15.2	Header <execution_policy> synopsis
20.15.3	Execution policy type trait
20.15.4	Sequential execution policy
20.15.5	Parallel execution policy
20.15.6	Parallel+Vector execution policy
20.15.7	Execution policy objects
18	Language support library
18.8	Exception handling
18.8.NaN	Class exception_list
25	Algorithms library
25.NaN	Parallel algorithms
25.NaN	Terms and definitions
25.NaN	Requirements on user-provided function objects
25.NaN	Effect of execution policies on algorithm execution
25.NaN	Parallel algorithm exceptions
25.NaN	ExecutionPolicy algorithm overloads
25	Algorithms library
25.2	Non-modifying sequence operations
25.2.4	For each
26	Numerics library
26.2	Definitions
26	Numerics library
26.7	Generalized numeric operations
26.7.1	Reduce
26.7.2	Exclusive scan
26.7.3	Inclusive scan
26.7.4	Transform reduce
26.7.5	Transform exclusive scan
26.7.6	Transform inclusive scan

Introducing Execution Policies

Add the following entry to Table 44:

20.15	Execution policies	<execution_policy>
-------	--------------------	--------------------

and corresponding entry <execution_policy> to Table 14 and update the reported number of headers.

Add a new subclause to Clause 20:

20.15 Execution policies

[execpol]

20.15.1 In general

[execpol.general]

This subclause describes classes that are *execution policy* types. An object of an execution policy type indicates the kinds of parallelism allowed in the execution of an algorithm and expresses the consequent requirements on the element access functions.

[*Example*:

```
using namespace std;
vector<int> v = ...

// standard sequential sort
sort(v.begin(), v.end());

// explicitly sequential sort
sort(sequential, v.begin(), v.end());

// permitting parallel execution
sort(par, v.begin(), v.end());

// permitting vectorization as well
sort(par_vec, v.begin(), v.end());
```

— *end example*]

[*Note*: Because different parallel architectures may require idiosyncratic parameters for efficient execution, implementations may provide additional execution policies to those described in this standard as extensions. — *end note*]

20.15.2 Header <execution_policy> synopsis

[execpol.synopsis]

```
namespace std {
    // 20.15.3, Execution policy type trait
    template<class T> struct is_execution_policy;
    template<class T> constexpr bool is_execution_policy_v = is_execution_policy<T>::value;

    // 20.15.4, Sequential execution policy
    class sequential_execution_policy;

    // 20.15.5, Parallel execution policy
    class parallel_execution_policy;

    // 20.15.6, Parallel+Vector execution policy
    class parallel_vector_execution_policy;
}
```

20.15.3 Execution policy type trait

[execpol.type]

```
template<class T> struct is_execution_policy { see below };

is_execution_policy can be used to detect execution policies for the purpose of excluding function signatures from otherwise ambiguous overload resolution participation.

is_execution_policy<T> shall be a UnaryTypeTrait with a BaseCharacteristic of true_type if T is the type of a standard or implementation-defined execution policy, otherwise false_type.
```

[*Note*: This provision reserves the privilege of creating non-standard execution policies to the library implementation. — *end note*]

The behavior of a program that adds specializations for is_execution_policy is undefined.

20.15.4 Sequential execution policy

[execpol.seq]

```
class sequential_execution_policy { unspecified };

The class sequential_execution_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and require that a parallel algorithm's execution may not be parallelized.
```

20.15.5 Parallel execution policy

[execpol.par]

```
class parallel_execution_policy { unspecified };

The class parallel_execution_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized.
```

20.15.6 Parallel+Vector execution policy

[execpol.vec]

```
class parallel_vector_execution_policy { unspecified };

The class parallel_vector_execution_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized and vectorized.
```

```
constexpr sequential_execution_policy      sequential{ unspecified };
constexpr parallel_execution_policy        par{ unspecified };
constexpr parallel_vector_execution_policy par_vec{ unspecified };
```

The header <execution_policy> declares global objects associated with each type of execution policy.

Introducing exception_list

Add the following entry to Table 41:

18.8.?	Exception list	<exception_list>
--------	----------------	------------------

and corresponding entry <exception_list> to Table 14 and adjust the reported number of headers.

Add a new section to Clause 18 after exception_ptr:

```
namespace std {

    class exception_list : public exception
    {
    public:
        typedef unspecified iterator;

        size_t size() const noexcept;
        iterator begin() const noexcept;
        iterator end() const noexcept;

        virtual const char* what() const noexcept;
    };
}
```

The class `exception_list` owns a sequence of `exception_ptr` objects. Parallel algorithms use the `exception_list` to communicate uncaught exceptions encountered during parallel execution to the caller of the algorithm.

The type `exception_list::iterator` is a constant iterator that meets the requirements of `ForwardIterator`.

`size_t size() const noexcept;`

- ⁴ *Returns:* The number of `exception_ptr` objects contained within the `exception_list`.
- ⁵ *Complexity:* Constant time.

`iterator begin() const noexcept;`

- ⁷ *Returns:* An iterator referring to the first `exception_ptr` object contained within the `exception_list`.

`iterator end() const noexcept;`

- ⁹ *Returns:* An iterator that is the past-the-end value of the owned sequence.

`virtual const char* what() const noexcept;`

- ¹¹ *Returns:* An implementation-defined NTBS.

Introducing general parallel algorithms content

Insert the following subclause after Section 25.1:

This section describes components that C++ programs may use to perform operations on containers and other sequences in parallel.

A *parallel algorithm* is a function template listed in this standard with a template parameter named `ExecutionPolicy`.

Parallel algorithms access objects indirectly accessible via their arguments by invoking the following functions:

- All operations of the categories of the iterators that the algorithm is instantiated with.
- Operations on those sequence elements that are required by its specification.
- User-provided function objects to be applied during the execution of the algorithm, if required by the specification.
- Operations on those function objects required by the specification. [*Note:* See (25.1). — *end note*]

These functions are herein called *element access functions*.

[*Example:* The `sort` function may invoke the following element access functions:

- Operations of the random-access iterator of the actual template argument, as per 24.2.7, as implied by the name of the template parameter `RandomAccessIterator`.
- The `swap` function on the elements of the sequence (as per 25.4.1.1 [sort]/2).
- The user-provided `Compare` function object.

— *end example*]

Function objects passed into parallel algorithms as objects of type `Predicate`, `BinaryPredicate`, `Compare`, and `BinaryOperation` shall not directly or indirectly modify objects via their arguments.

Parallel algorithms have template parameters named `ExecutionPolicy` which describe the manner in which the execution of these algorithms may be parallelized and the manner in which they apply the element access functions.

The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `sequential_execution_policy` are indeterminately sequenced (1.9) [intro.execution] in the calling thread.

The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `parallel_execution_policy` are permitted to execute in either the invoking thread or in a thread implicitly created by the library to support parallel algorithm execution. Any such invocations executing in the same thread are indeterminately sequenced (1.9) [intro.execution] with respect to each other. [*Note:* It is the caller's responsibility to ensure correctness, for example that the invocation does not introduce data races or deadlocks. — *end note*]

[*Example:*

```
int a[] = {0,1};
std::vector<int> v;
std::for_each(std::par, std::begin(a), std::end(a), [&](int i) {
    v.push_back(i*2+1); // Error: data race
});
```

The program above has a data race because of the unsynchronized access to the container `v`. — *end example*]

[*Example:*

```
std::atomic<int> x{0};
int a[] = {1,2};
std::for_each(std::par, std::begin(a), std::end(a), [&](int) {
    x.fetch_add(1, std::memory_order_relaxed);
    // spin wait for another iteration to change the value of x
    while (x.load(std::memory_order_relaxed) == 1) { } // Error: assumes execution order
});
```

The above example depends on the order of execution of the iterations, and is therefore undefined (may deadlock). — *end example*]

[*Example:*

```
int x = 0;
std::mutex m;
int a[] = {1,2};
std::for_each(std::par, std::begin(a), std::end(a), [&](int) {
    std::lock_guard<mutex> guard(m);
    ++x;
});
```

The above example synchronizes access to object `x` ensuring that it is incremented correctly. — *end example*]

The invocations of element access functions in parallel algorithms invoked with an execution policy of type `parallel_vector_execution_policy` are permitted to execute in an unordered fashion in unspecified threads, and unsequenced with respect to one another within each thread. [*Note:* This means that multiple function object invocations may be interleaved on a single thread, which overrides the usual guarantee from 1.9 [intro.execution] that function executions do not interleave with one another. — *end note*]

Since `parallel_vector_execution_policy` allows the execution of element access functions to be interleaved on a single thread, synchronization, including the use of mutexes, risks deadlock. Thus the synchronization with `parallel_vector_execution_policy` is restricted as follows:

A standard library function is *vectorization-unsafe* if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function. Vectorization-unsafe standard library functions may not be invoked by user

code called from `parallel_vector_execution_policy` algorithms.

[*Note:* Implementations must ensure that internal synchronization inside standard library routines does not induce deadlock. — *end note*]

[*Example:*

```
int x = 0;
std::mutex m;
int a[] = {1,2};
std::for_each(std::par_vec, std::begin(a), std::end(a), [&](int) {
    std::lock_guard<mutex> guard(m); // Error: lock_guard constructor calls m.lock()
    ++x;
});
```

The above program has undefined behavior because the applications of the function object are not guaranteed to run on different threads. This may result in two consecutive calls to `m.lock()` on the same thread, which may deadlock. — *end example*]

[*Note:* The semantics of the `parallel_execution_policy` or the `parallel_vector_execution_policy` invocation allow the implementation to fall back to sequential execution if the system cannot parallelize an algorithm invocation due to lack of resources. — *end note*]

The semantics of parallel algorithms invoked with an execution policy object of implementation-defined type are implementation-defined.

25.NaN.4 Parallel algorithm exceptions

[algorithms.parallel.exceptions]

During the execution of a parallel algorithm, if temporary memory resources are required for parallelization and none are available, the algorithm throws a `bad_alloc` exception.

During the execution of a parallel algorithm, if the invocation of an element access function exits via an uncaught exception, the behavior of the program is determined by the type of execution policy used to invoke the algorithm:

- If the execution policy object is of type `parallel_vector_execution_policy`, `terminate()` is called.
 - If the execution policy object is of type `sequential_execution_policy` or `parallel_execution_policy`, the execution of the algorithm exits via an exception. The exception will be an `exception_list` containing all uncaught exceptions thrown during the invocations of element access functions, or optionally the uncaught exception if there was only one.
- [*Note:* For example, when `for_each` is executed sequentially, if an invocation of the user-provided function object throws an exception, `for_each` can exit via the uncaught exception, or throw an `exception_list` containing the original exception. — *end note*]
- [*Note:* These guarantees imply that, unless the algorithm has failed to allocate memory and exits via `bad_alloc`, all exceptions thrown during the execution of the algorithm are communicated to the caller. It is unspecified whether an algorithm implementation will "forge ahead" after encountering and capturing a user exception. — *end note*]
- [*Note:* The algorithm may exit via the `bad_alloc` exception even if one or more user-provided function objects have exited via an exception. For example, this can happen when an algorithm fails to allocate memory while creating or adding elements to the `exception_list` object. — *end note*]
- If the execution policy object is of any other type, the behavior is implementation-defined.

25.NaN.5 ExecutionPolicy algorithm overloads

[algorithms.parallel.overloads]

Parallel algorithms are algorithm overloads. Each parallel algorithm overload has an additional template type parameter named `ExecutionPolicy`, which is the first template parameter. Additionally, each parallel algorithm overload has an additional function parameter of type `ExecutionPolicy&&`, which is the first function parameter. [*Note:* Not all algorithms have parallel algorithm overloads. — *end note*]

Unless otherwise specified, the semantics of `ExecutionPolicy` algorithm overloads are identical to their overloads without.

Parallel algorithms shall not participate in overload resolution unless `is_execution_policy_v<decay_t<ExecutionPolicy>>` is true.

Table 1 — Table of parallel algorithms			
<code>adjacent_difference</code>	<code>adjacent_find</code>	<code>all_of</code>	<code>any_of</code>
<code>copy</code>	<code>copy_if</code>	<code>copy_n</code>	<code>count</code>
<code>count_if</code>	<code>equal</code>	<code>exclusive_scan</code>	<code>fill</code>
<code>fill_n</code>	<code>find</code>	<code>find_end</code>	<code>find_first_of</code>
<code>find_if</code>	<code>find_if_not</code>	<code>for_each</code>	<code>for_each_n</code>
<code>generate</code>	<code>generate_n</code>	<code>includes</code>	<code>inclusive_scan</code>
<code>inner_product</code>	<code>inplace_merge</code>	<code>is_heap</code>	<code>is_heap_until</code>
<code>is_partitioned</code>	<code>is_sorted</code>	<code>is_sorted_until</code>	<code>lexicographical_compare</code>
<code>max_element</code>	<code>merge</code>	<code>min_element</code>	<code>minmax_element</code>
<code>mismatch</code>	<code>move</code>	<code>none_of</code>	<code>nth_element</code>
<code>partial_sort</code>	<code>partial_sort_copy</code>	<code>partition</code>	<code>partition_copy</code>
<code>reduce</code>	<code>remove</code>	<code>remove_copy</code>	<code>remove_copy_if</code>
<code>remove_if</code>	<code>replace</code>	<code>replace_copy</code>	<code>replace_copy_if</code>
<code>replace_if</code>	<code>reverse</code>	<code>reverse_copy</code>	<code>rotate</code>
<code>rotate_copy</code>	<code>search</code>	<code>search_n</code>	<code>set_difference</code>
<code>set_intersection</code>	<code>set_symmetric_difference</code>	<code>set_union</code>	<code>sort</code>
<code>stable_partition</code>	<code>stable_sort</code>	<code>swap_ranges</code>	<code>transform</code>
<code>transform_exclusive_scan</code>	<code>transform_inclusive_scan</code>	<code>transform_reduce</code>	<code>uninitialized_copy</code>
<code>uninitialized_copy_n</code>	<code>uninitialized_fill</code>	<code>uninitialized_fill_n</code>	<code>unique</code>
<code>unique_copy</code>			

[*Note:* Note to the editor: this table should not be introduced into the standard. This is the list of all algorithms, including new algorithms introduced by this proposal, whose header synopses and definitions should be augmented with parallel algorithm overloads. — *end note*]

Modifications to header synopses

For each algorithm listed in [Table 1](#), add the signature of a parallel algorithm overload to the corresponding synopsis in Clause 20, Clause 25, or Clause 26.

New algorithms should be placed at the editor's discretion, after considering the following suggestion:

- `for_each_n` should follow `for_each`

- `reduce` should follow `accumulate`
- `transform_reduce` should follow `reduce`
- `exclusive_scan` should follow `partial_sum`
- `inclusive_scan` should follow `exclusive_scan`
- `transform_exclusive_scan` should follow `inclusive_scan`
- `transform_inclusive_scan` should follow `transform_exclusive_scan`

Specify new algorithms

Add `for_each` with `ExecutionPolicy`, `sequential_for_each_n`, and `for_each_n` with `ExecutionPolicy` to (25.2.4):

```
template<class ExecutionPolicy,
         class InputIterator, class Function>
void for_each(ExecutionPolicy&& exec,
              InputIterator first, InputIterator last,
              Function f);
```

² *Requires:* Function shall meet the requirements of CopyConstructible.

³ *Effects:* Applies `f` to the result of dereferencing every iterator in the range `[first,last)`. [*Note:* If the type of `first` satisfies the requirements of a mutable iterator, `f` may apply nonconstant functions through the dereferenced iterator. — *end note*]

⁴ *Complexity:* Applies `f` exactly `last - first` times.

⁵ *Remarks:* If `f` returns a result, the result is ignored.

⁶ *Notes:* Does not return a copy of its `Function` parameter, since parallelization may not permit efficient state accumulation.

```
template<class InputIterator, class Size, class Function>
InputIterator for_each_n(InputIterator first, Size n,
                        Function f);
```

⁸ *Requires:* Function shall meet the requirements of MoveConstructible [*Note:* Function need not meet the requirements of CopyConstructible. — *end note*]

⁹ *Requires:* `n >= 0`.

¹⁰ *Effects:* Applies `f` to the result of dereferencing every iterator in the range `[first,first + n)` in order. [*Note:* If the type of `first` satisfies the requirements of a mutable iterator, `f` may apply nonconstant functions through the dereferenced iterator. — *end note*]

¹¹ *Returns:* `first + n`.

¹² *Remarks:* If `f` returns a result, the result is ignored.

```
template<class ExecutionPolicy,
         class InputIterator, class Size, class Function>
InputIterator for_each_n(ExecutionPolicy&& exec,
                        InputIterator first, Size n,
                        Function f);
```

¹⁴ *Requires:* Function shall meet the requirements of CopyConstructible.

¹⁵ *Requires:* `n >= 0`.

¹⁶ *Effects:* Applies `f` to the result of dereferencing every iterator in the range `[first,first + n)`. [*Note:* If the type of `first` satisfies the requirements of a mutable iterator, `f` may apply nonconstant functions through the dereferenced iterator. — *end note*]

¹⁷ *Returns:* `first + n`.

¹⁸ *Remarks:* If `f` returns a result, the result is ignored.

Introducing numeric parallel algorithms definitions

Insert the following entry to Table 113:

26.2	Definitions
------	-------------

Insert the following subclause to Clause 26:

Define *GENERALIZED_SUM*(op, a1, ..., aN) as follows:

- a1 when N is 1
- op(*GENERALIZED_SUM*(op, b1, ..., bK), *GENERALIZED_SUM*(op, bM, ..., bN)) where
 - b1, ..., bN may be any permutation of a1, ..., aN and
 - 1 < K+1 = M ≤ N.

Define *GENERALIZED_NONCOMMUTATIVE_SUM*(op, a1, ..., aN) as follows:

- a1 when N is 1
- op(*GENERALIZED_NONCOMMUTATIVE_SUM*(op, a1, ..., aK), *GENERALIZED_NONCOMMUTATIVE_SUM*(op, aM, ..., aN) where 1 < K+1 = M ≤ N.

Specify new numeric algorithms

Add reduce, exclusive_scan, inclusive_scan, transform_reduce, transform_exclusive_scan, and transform_inclusive_scan to Clause 26.7.

```
template<class InputIterator>
    typename iterator_traits<InputIterator>::value_type
        reduce(InputIterator first, InputIterator last);

2 Effects: Equivalent to return reduce(first, last, typename iterator_traits<InputIterator>::value_type{}).
```

```
template<class InputIterator, class T>
    T reduce(InputIterator first, InputIterator last, T init);

4 Effects: Equivalent to return reduce(first, last, init, plus<>()).
```

```
template<class InputIterator, class T, class BinaryOperation>
    T reduce(InputIterator first, InputIterator last, T init,
        BinaryOperation binary_op);

6 Returns: GENERALIZED_SUM(binary_op, init, *i, ...) for every i in [first, last).
7 Requires: binary_op shall neither invalidate iterators or subranges, nor modify elements in the range [first,last).
8 Complexity: O(last - first) applications of binary_op.
9 Notes: The difference between reduce and accumulate is that reduce applies binary_op in an unspecified order, which yields a non-deterministic result for non-associative or non-commutative binary_op such as floating-point addition.
```

```
template<class InputIterator, class OutputIterator, class T>
    OutputIterator exclusive_scan(InputIterator first, InputIterator last,
        OutputIterator result,
        T init);

2 Effects: Equivalent to return exclusive_scan(first, last, result, init, plus<>()).
```

```
template<class InputIterator, class OutputIterator, class T, class BinaryOperation>
    OutputIterator exclusive_scan(InputIterator first, InputIterator last,
        OutputIterator result,
        T init, BinaryOperation binary_op);

4 Effects: Assigns through each iterator i in [result,result + (last - first)) the value of
    • init, if i is result, otherwise
    • GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *j, ...) for every j in [first,first + (i - result) - 1).

5 Returns: The end of the resulting range beginning at result.
6 Requires: binary_op shall neither invalidate iterators or subranges, nor modify elements in the ranges [first,last) or [result,result + (last - first)).
7 Complexity: O(last - first) applications of binary_op.
8 Notes: The difference between exclusive_scan and inclusive_scan is that exclusive_scan excludes the ith input element from the ith sum. If binary_op is not mathematically associative, the behavior of exclusive_scan may be non-deterministic.
9 Remarks: result may be equal to first.
```

```
template<class InputIterator, class OutputIterator>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
        OutputIterator result);

2 Effects: Equivalent to return inclusive_scan(first, last, result, plus<>()).
```

```
template<class InputIterator, class OutputIterator, class BinaryOperation>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
        OutputIterator result,
        BinaryOperation binary_op);

template<class InputIterator, class OutputIterator, class BinaryOperation>
    OutputIterator inclusive_scan(InputIterator first, InputIterator last,
        OutputIterator result,
        BinaryOperation binary_op, T init);

4 Effects: Assigns through each iterator i in [result,result + (last - first)) the value of GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, *j, ...) for every j in [first,first + (i - result)) or GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *j, ...) for every j in [first,first + (i - result)) if init is provided.
5 Returns: The end of the resulting range beginning at result.
6 Requires: binary_op shall not invalidate iterators or subranges, nor modify elements in the ranges [first,last) or [result,result + (last - first)).
7 Complexity: O(last - first) applications of binary_op.
8 Remarks: result may be equal to first.
9 Notes: The difference between exclusive_scan and inclusive_scan is that inclusive_scan includes the ith input element in the ith sum. If binary_op is not mathematically associative, the behavior of inclusive_scan may be non-deterministic.
```

```
template<class InputIterator, class UnaryFunction, class T, class BinaryOperation>
    T transform_reduce(InputIterator first, InputIterator last,
                      UnaryOperation unary_op, T init, BinaryOperation binary_op);
```

- ² *Returns:* *GENERALIZED_SUM*(binary_op, init, unary_op(*i), ...) for every i in [first,last).
- ³ *Requires:* Neither unary_op nor binary_op shall invalidate subranges, or modify elements in the range [first,last)
- ⁴ *Complexity:* O(last - first) applications each of unary_op and binary_op.
- ⁵ *Notes:* transform_reduce does not apply unary_op to init.

26.7.5 Transform exclusive scan

[\[transform.exclusive.scan\]](#)

```
template<class InputIterator, class OutputIterator,
        class UnaryOperation,
        class T, class BinaryOperation>
    OutputIterator transform_exclusive_scan(InputIterator first, InputIterator last,
                                          OutputIterator result,
                                          UnaryOperation unary_op,
                                          T init, BinaryOperation binary_op);
```

- ² *Effects:* Assigns through each iterator i in [result,result + (last - first)) the value of
 - init, if i is result, otherwise
 - *GENERALIZED_NONCOMMUTATIVE_SUM*(binary_op, init, unary_op(*j), ...) for every j in [first,first + (i - result) - 1).
- ³ *Returns:* The end of the resulting range beginning at result.
- ⁴ *Requires:* Neither unary_op nor binary_op shall invalidate iterators or subranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).
- ⁵ *Complexity:* O(last - first) applications each of unary_op and binary_op.
- ⁶ *Remarks:* result may be equal to first.
- ⁷ *Notes:* The difference between transform_exclusive_scan and transform_inclusive_scan is that transform_exclusive_scan excludes the ith input element from the ith sum. If binary_op is not mathematically associative, the behavior of transform_exclusive_scan may be non-deterministic. transform_exclusive_scan does not apply unary_op to init.

26.7.6 Transform inclusive scan

[\[transform.inclusive.scan\]](#)

```
template<class InputIterator, class OutputIterator,
        class UnaryOperation,
        class BinaryOperation>
    OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                          OutputIterator result,
                                          UnaryOperation unary_op,
                                          BinaryOperation binary_op);

template<class InputIterator, class OutputIterator,
        class UnaryOperation,
        class BinaryOperation, class T>
    OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                          OutputIterator result,
                                          UnaryOperation unary_op,
                                          BinaryOperation binary_op, T init);
```

- ² *Effects:* Assigns through each iterator i in [result,result + (last - first)) the value of *GENERALIZED_NONCOMMUTATIVE_SUM*(binary_op, unary_op(*j), ...) for every j in [first,first + (i - result)) or *GENERALIZED_NONCOMMUTATIVE_SUM*(binary_op, init, unary_op(*j), ...) for every j in [first,first + (i - result)) if init is provided.
- ³ *Returns:* The end of the resulting range beginning at result.
- ⁴ *Requires:* Neither unary_op nor binary_op shall invalidate iterators or subranges, or modify elements in the ranges [first,last) or [result,result + (last - first)).
- ⁵ *Complexity:* O(last - first) applications each of unary_op and binary_op.
- ⁶ *Remarks:* result may be equal to first.
- ⁷ *Notes:* The difference between transform_exclusive_scan and transform_inclusive_scan is that transform_inclusive_scan includes the ith input element in the ith sum. If binary_op is not mathematically associative, the behavior of transform_inclusive_scan may be non-deterministic. transform_inclusive_scan does not apply unary_op to init.